Code-Layout, Readability and Reusability

- Date: 15/10/2026
- Team ID: SWTID-2026-4348
- Team Size: 1
- Team Leader:Naveen
- Project Name: AI FitTrack
- **Maximum Marks:** 3 Marks
-

Maximum Marks: 5 Marks

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standardized 2-space JavaScript formatting enforced across all controllers and services.
2	Proper File Structure	Files and folders are logically organized	Yes	Modular MVC layout (src/config, controllers, middleware, models, routes, services).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Descriptive variable identifiers used (e.g., workoutName, caloriesBurned, averageDuration).
4	Function / Method Names	Functions are descriptively named	Yes	Verb-driven descriptive names (e.g., generateWorkoutRecommendation, matchPassword).
5	Code Comments	Inline and block comments explain logic	Yes	Detailed block comments for pre-save hooks, JWT checks, and Gemini service calls.
6	Modular Design	Code is split into reusable functions/modules	Yes	Business logic, authentication middleware, and AI APIs isolated in dedicated modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	DRY principle maintained using centralized middleware (auth.js, errorHandler.js).
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Asynchronous try-catch blocks and centralized JSON error handler responses.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	connectDB	Node.js / Mongoose	Database initialization on	High

			application startup (server.js)	
2	protect (Auth Middleware)	Express.js / JWT	Applied across all secure Workout and AI endpoints (workoutRoutes.js, aiRoutes.js)	High
3	geminiService	Node.js / @google/genai	Called inside aiController.js for custom workout generation and stats analytics	High
4	matchPassword Hook	Mongoose / bcryptjs	Reused inside authController.js during user login verification	High
5	errorHandler Middleware	Express.js	Centralized error formatting layer applied globally across Express routes	High

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5 / 5	Strict MVC separation ensures clean, maintainable folder hierarchy.
Readability	5 / 5	Clear naming conventions and consistent formatting throughout all JavaScript files.
Reusability	5 / 5	Service-oriented layer isolates Gemini AI integration and JWT authentication cleanly.
Documentation / Comments	4.5 / 5	Comprehensive inline comments and clear error logging in service handlers.
Overall Score	4.9 / 5	Excellent enterprise-grade RESTful API backend structure.

Note: This Word version contains editable text and tables.